
Lab Assignment 5

In this assignment you are to analyze the performance of a C program that computes the number of prime numbers up to some value, n .

1. Login to prospero via ssh (`ssh <yourusername>@prospero.cs.wit.edu`)
2. Use **MPI** to parallelize the application.
3. Use `-lm` in your in your linking step after the executable in your makefile so that math library functions can be used.
4. Verify that the code still produces the correct result.
5. Do **NOT** run the program on the login node, create a batch script and submit to the cluster.
6. Create a **PDF** file with a write-up that includes the content described in the Expected Results section.
7. Upload your report **PDF** to Brightspace in the appropriate location.

Why:

Most problems are not easily parallelized. For example, in that code that you are given, the algorithm for computing the number of primes in a region is not load balanced. Thus, to get a reasonable speedup, you will need to ensure that all ranks have approximately the same amount of work. Load balancing issues are common in real world applications and learning to solve them is an important tool to learn.

How:

Since you are using MPI to parallelize this application, you **must use a full Master/Worker algorithm** to solve the load balancing problem. The outline for the algorithm is:

Master:

- **Send** a small range of values to each worker, the “chunk” size.
- Wait to **Recv** the results from anyone.
- Add to the running total.
- **IF** there is no more work to do, send an invalid range to the worker.
- Send a new range back to the worker.
- **IF** all workers are finished, proceed to the end of the program.
- Repeat.

Worker:

- **Recv** a small range of work from master.
- **IF** master gives an invalid range, continue to the end of the program.
- Compute the number of primes in this range.
- **Send** the number of primes back to master.
- Repeat.

This is similar to what we worked on in lecture for computing the integral. Use the lecture as a starting point for this assignment.

Expected Results:

Use the MPI algorithm to create three different plots to include in your PDF:

1. A plot that shows the time vs. the amount of work assigned to each rank (the chunk size). Time is on the y-axis and the amount of work, or “chunk” size, is on the x-axis. This will show you the ideal value for the amount work that should be given to each rank to get the best performance. Use chunk sizes over a large range, for example, 1-1 million in powers of 10. This will require a log x-axis.
2. A strong scaling plot of a load imbalanced version. To do this, choose an amount of work for each rank that evenly splits the number of iterations (or as close as possible). This will approximate a naïve implementation.
3. A strong scaling plot of the load balanced using the best chunk size you found in plot 1.